01/04/2026

Numerical Differential Equations

Luca Persichetti

Dipartimento di Matematica, Università degli Studi di Bologna

---

We use Python for this project to avoid dependence on proprietary software such as MATLAB and because we think that open-source software is a great paradigm for scientific work. The main difference is that the array indexing starts from 0 in python, while it starts from 1 in MATLAB.
Section 1 reviews the computational exercises covered during the course.
In the second section we explain our implementation of a Reaction-Diffusion model in Alzheimer's disease: describing the model in the first paragraph and then explaining our solution of the assignments in section 2.2, 2.3 and 2.4.
The files are organized in folders: the laboratories are in the 'PDE' folder and the project files are in the 'A' folder inside the one metioned. For the project we also needed some matlab file that are in the 'MAT' folder.
All the python functions used are stored in $\star$.py files while the testing scripts are inside python notebooks ($\star$.ipynb).

Contents

1 Finite Difference (FD) Approximation of Differential Operators **1**
1.1 Review of ordinary differential equations (ODEs) discretization . . . 1
1.2 Elliptic operators . . . 3
1.2.1 Iterative solvers . . . 4
1.2.2 Robin's BC . . . 4
1.2.3 Convection and reaction . . . 5
1.2.4 Nonlinear elliptic equations . . . 5
1.3 Parabolic operators . . . 6
1.4 Hyperbolic operators . . . 7
1.4.1 Burger's and wave equations . . . 7

2 Reaction-Diffusion Model in Alzheimer's disease **8**
2.1 Model description . . . 8
2.2 Stationary Model - FD . . . 8
2.3 Stationary Model - Finite Elements Method (FEM) . . . 9
2.4 Full Model - FEM and FD . . . 10

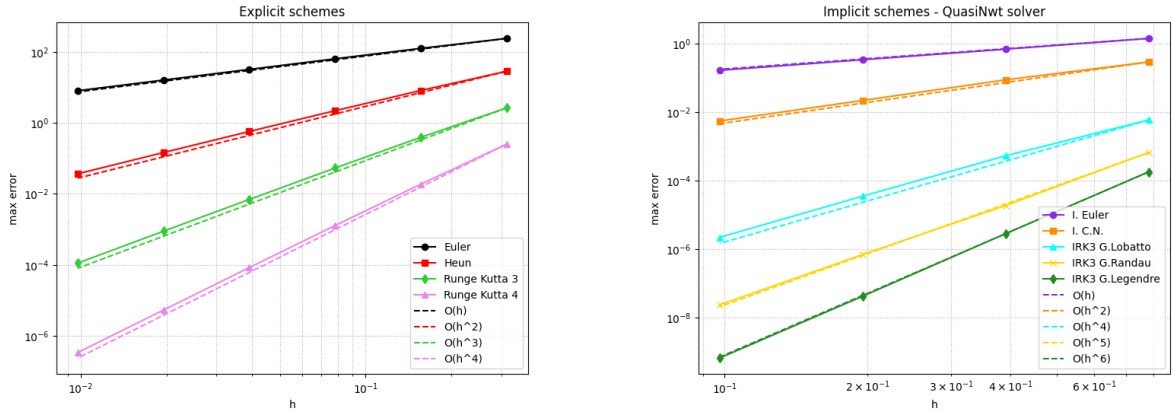
3 Some implementation details and conclusions **10**

---

1. Finite Difference (FD) Approximation of Differential Operators

1.1. Review of ordinary differential equations (ODEs) discretization

Since we worked using the python language here we review some Onumerical integration schemes for ODES and fix notation and implementation conventions used in the rest of the following pages. Our



(a) (b)

Figure 1: Convergence plot ODEs methods.

first goal is to approximate $\mathbf{u} = \mathbf{u}(t) = [u_1(t), ..., u_d(t)] : \Omega \to \mathbb{R}^d$ defined on $\Omega = [t_0, t_f] \subset \mathbb{R}$ as a solution of the (d-dimensional) initial value problem (IVP):

$$\begin{cases} \mathbf{u}_t = \mathbf{f}(t, \mathbf{u}), t \in (t_0, t_f] \\ \mathbf{u}(t_0) = \mathbf{u}_0 \end{cases}$$

Some notation: from now on, by the discretization or numerical approximation of $\mathbf{u}(t)$ we mean its sampling on a discrete time grid. To do that we discretize the time domain using $N+1$ uniformly spaced points: $\bar{t} = \begin{bmatrix} t_0 \\ t_1 \\ \vdots \\ t_N \end{bmatrix}$. By this notation we mean: let $h = (t_f - t_0)/N$and $t_i = t_0 + hi$, $(\implies t_N = t_f)$. Let for example $u = u(t)$ a (one dimensional) function we need to approximate in $[t_0, t_f]$; from now on we denote $\overline{u}$ the sampling of u on the time grid $\bar{t}$ meaning: $\overline{u} = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} u(t_0) \\ u(t_1) \\ \vdots \\ u(t_N) \end{bmatrix}$. The variable t can be thought of as a spatial variable, called x instead of a time one and in this case we subdivide the space in $N + 2$ points because of symmetry.

In the file `meth1_E.py` are defined some explicit schemes to solve IVPs translated from the MATLAB exercises solved during my first year studying applied mathematics at University of Bologna. Explicit Euler method `Expl_Euler`, Heun `Expl_Heun` and iterative Runge Kutta of order 3/4 `ERK3`, `ERK4` are present. These methods are tested in the notebook `testODE_E.ipynb` for the solution of a constant population Suscettibles-Infecrtives-Recovered (SIR) model in epidemiology. To compute the solution we need the `numpy` module to be imported; to plot we use `matplotlib.pyplot` and to verify the theoretical convergence we import the `solve_ivp` module from the `scipy.integrate` library to compute a reference solution. We will test this integrated solver better in a few lines. The theoretical convergence is matched as shown in Figure 1. For implicit solvers we need some non-linear solver: a `newton_solver` and a `quasi_newton_solver` with damping parameter are defined in `quasi_newton.py`. These solvers are used in the implicit methods defined in `meth2_I.py`. Defined here we have implicit Euler `impl_euler`, Cranck Nicolson `impl_CN` and 3-stages implicit Runge Kutta schemes using Gauss Legendre, Gauss Randau or Gauss Lobatto quadrature formulas `IRK3_solver(method='...')`. The functions named adding a `_quasi` at the end of the names above use an approximation of the Jacobian that it's not then needed as input. As above, these methods were tested in `testODE_I.ipynb` over an example and expected convergence is achieved as shown in Figure 1. The example is the FitzHugh-Nagumo model of the excitation dynamics of a neuron. Finally to test `scipy` integrated solver `solve_ivp` we refer to the

Method	t_f	Runtime (s)	f evaluations	Peak Memory (KB)
RK23	100	39.8810	411701	44039.3
RK45	100	45.1908	732290	33493.5
DOP853	100	37.4046	647186	17301.0
LSODA	100000000	0.0486	1220	220.3
Radau	100000000	0.3819	2195	100.9
BDF	100000000	0.3666	1386	167.7

Table 1: Some tests of `solve_ivp`'s computational resources usage on Robertson's stiff example.

official SciPy documentation and import the modules `time` and `tracemalloc` to access the system time and memory usage. The test example will be the stiff chemical kinetics example by Robertson from the Wikipedia page of stiff problems. All the methods use adaptive stepsize selection (h fixed) and can produce a dense output using interpolation tecniques. As expected the explicit methods 'RK23','RK45' and 'DOP853' can't solve the problem in an acceptable amount of time for $t_f > 100$ while the implicit methods 'Randau', 'BDF' and 'LSODA' can reach far higher t_f. Results are shown in Table 1.

1.2. Elliptic operators

Here we think the variable of the system as spatial $x, (x, y), (x, y, z), ...$ instead of temporal but we keep the notation $\overline{t}, \overline{x}$ meaning the sampling on a uniform grid. To fix notations firstly let's consider the 1-dimensional boundary value problem (BVP) involving the Poisson equation and homogeneus Dirichlet boundary conditions (BC):

$$\begin{cases} -u_{xx}(x) = f(x), \ x \in \Omega = \\ u = 0, \ x \in \partial\Omega \end{cases}$$

We approximate $u_{xx}(x_i)$ by central finite differences:

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h_x^2} = f(x_i) \quad \forall i = 0, ..., N+1$$

.Note for symmetry it is cleaner to subdivide the interval in $N+2$ parts using index 0 and $N+1$ for boundaries. This way we obtain a set of $N+2$ linear equations in $u_i \approx u(x_i) \forall i$ written matrix form:

$$A\overline{u} = \overline{f} = \qquad \text{, where } A = D_{xx} = \frac{1}{h^2} \begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \ddots & \vdots \\ 0 & 1 & -2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{pmatrix},$$

This strategy scales beautifully with dimension. Indeed if we want to approximate the solution of the Poisson BVP in 2 or 3 dimensions (meaning that the equation is now $\Delta_\star u = f$: $\Delta_2 = \partial_{xx} + \partial_{yy}$ or $\Delta_3 = \partial_{xx} + \partial_{yy} + \partial_{zz}$ and Ω is a rectangle in 2D and a cuboid in 3D) can use the Kronecker product of matrices. To do that we consider the 5 points stencil in 2D and a 7 points one in the 3D case and thanks to Kronecker we obtain the clean formalism for the matrices

$$A = D_{xx} \otimes I_M + I_N \otimes D_{yy},$$

$$A_{3D} = D_{xx} \otimes I_M \otimes I_L + I_N \otimes D_{yy} \otimes I_L + I_N \otimes I_M \otimes D_{zz}$$

. The main difference with MATLAB is that our indexing convention fis implemented using the meshgrid function with indexing='ij'. We note that the dimension of the linear system we need to

N_x	**Meth.**	**Runtime 2D(s)**	**Runtime 3D(s)**	**Memory 2D(KB)**	**Memory 3D(KB)**
4	S.O.R.	0.17	0.66	1.1	1.5
8	S.O.R.	0.66	7.70	1.5	5.8
16	S.O.R.	2.93	84.38	3.0	34.0
256	bicgstab	0.003	0.78	>3e3	>9e5
256	gmres	0.008	1.48	>1e4	>3e6

Table 2: Computational resources usage. Splitting vs. descent methods on Poisson's example.

solve grows quite rapidly as the dimension gets higher so we need some smart methods to deal with it. This approximation is second order and the tests results are shown in Figure 2 (a).

1.2.1 Iterative solvers

The first 3 methods we explored are called splitting methods because the main idea is to split the system matrix into two or more matrices. The Jacobi method is based on splitting A into D the diagonal elements and O the off-diagonal ones. The Gauss-Seidel method starts by splitting A into the lower L and U upper diagonal elements. The successive over relaxation (SOR) method prescribe to split A into $D + L + U$ its diagonal, lower and upper diagonal (note that this L is not the same as above). It is an improvement of the Gauss-Seidel iteration and use a relaxation factor $\omega > 1$. All this methods are implementated in custom_solvers.py and they start the iterations with $x^0 = np.zeros(N)$. This methods are not suitable for real-world applications since the fastest of the three methods: SOR, uses a lot of computational resources even for small problems as shown in the first three lines of Table 2. The cause here is that we are using full (sometimes called dense) matrices while a sparse representation is the right choice in this case; we switched then to a sparse representation of the FD laplacian and tested some solvers found in the SCIPY module. As expected, using this solvers based on gradient descent we were able to solve the test problems with a finer grid in an acceptable amount of time. Results are shown in te last lines of Table 1. The structure of the matrix A is tridiagonal in the 1D case, symmetric Toeplitz in 2D and 3D. We exclude CG solver since the matix A will fail to be positive definite most of the times. Our tests show that MINRES uses a little more resources than BICGSTAB and GMRES. These tests were executed in the file testELL_dir_rob.ipynb where we checked convergence and computational efficiency.

1.2.2 Robin's BC

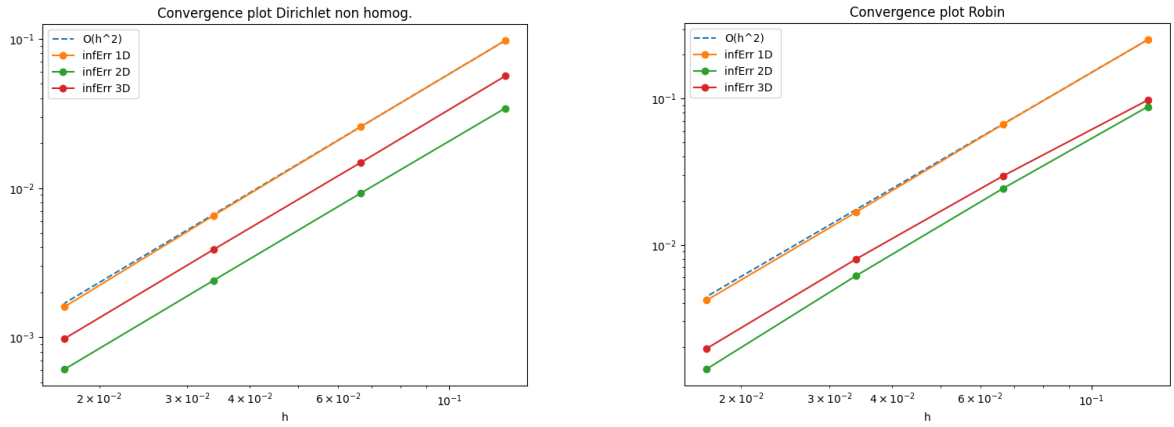
In order to capture more interesting behaviours of the dynamical systems in the applications we choose to implement BC of the form:

$$\alpha \frac{\partial u}{\partial n}(\mathbf{x}) + \beta u(\mathbf{x}) = \tilde{g}, \qquad \forall x \in \partial\Omega$$

. Here $\mathbf{n}$ mean the normal to the corresponding boundary and we denoted the coefficients α, β with bold font meaning that they are a structure depending on dimension: in 1D there are two alphas α_L, α_R, two βs and two $\tilde{g}$s, in 2D there are four of every one of them (called L, R, B, T to indicate the corresponding edge) and six in the 3D case (L, R, B, T, D, U). This way we can model a fixed value edge using $\alpha_* = 0$ (Dirichlet non-homogeneous BCs), a prescribed flux through the edge using Neumann BCs: $\beta_* = 0$ and a convective behaving edge with both α, β not zero (Robin's BC). In the case of $\alpha \neq 0$, using the ghost node strategy, we first write the condition diving by α:

$$\frac{\partial u}{\partial n}(\mathbf{x}) + \mathbf{R}u(\mathbf{x}) = \mathbf{g} \quad R = \frac{\beta}{\alpha}, \mathbf{g} = \frac{\tilde{\mathbf{g}}}{\alpha}.$$

The implementation is done by modifying the first and last rows of our system's matrix and the corresponding value in the right hand side. This in the one dimensional case; in the two and three dimensional ones the modification of the matrix is performed before the Kronecker product is applied.



(a) (b)

Figure 2: Convergence plots: Poisson's equation.

Then the RHS is modified and, finally, the masks for the Dirichlet boundaries are computed and applied if needed. Some results are shown in Figure 2 (b) and we mention that our test problem is not well posed in the sense of Hadamard using homogeneus Neumann BCs.

1.2.3 Convection and reaction

To model some interesting biological behaviours, like the flux of a pollutant through a media for example, we need the ability to solve also PDEs containing convective terms: $... + b\partial_x u + ..+$ in 1D, $... + b_x\partial_x u + b_y\partial_y u + ...$ in 2D, $... + \mathbf{b}\nabla u + ...$, with $\mathbf{b} = [b_x, b_y, b_z]$ and ∇u meaning the 3D gradient. To discretize first derivatives we use central differences in in order to keep second order convergence rate. This computation is extended for dimensions grater than one using the Kronecker product as in the preceding section. The BCs are applied exactly using the strategy described above. Doing this we can discretize:

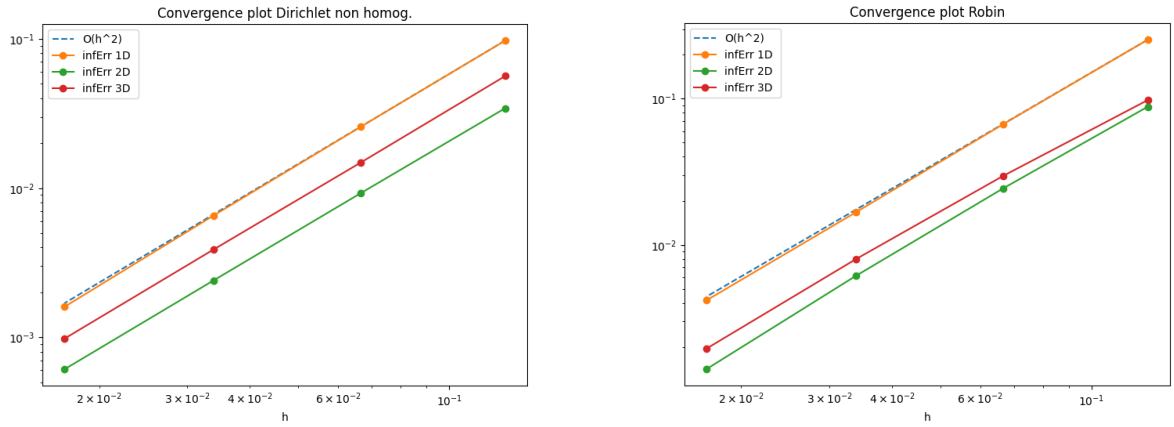
$$\Delta u + \mathbf{b}\nabla u = f \text{ as } (A + B)\overline{u} = \overline{f}$$

plus eventual modifications due to BCs. The discretization of the convection operator is performed by the FD_conv_homo() and FD_conv_sparse() functions in the FD.py file.
Another example of interesting feature we want to deal with in a PDE is the presence of a nonlinear reaction term: an example from biology could be the logistic growth of a population in an epidemiological model. We already solved a similar problem in ODE discretization so we use the same strategies also here: we'll use also this time the Newton and quasi-Newton solvers. These solvers need some modifications to deal with our sparse large matrices. Indeed in the inner iterations of the solvers we don't use the NUMPY standard solvers and we have to implement some sparse solvers. With some testing we discover that the direct solver SPSOLVE is faster for smaller matrices but needs to be abandoned for large ones. Test examples are worked out in the file testELL_cr.ipynb and results are shown in Figure 3.

1.2.4 Nonlinear elliptic equations

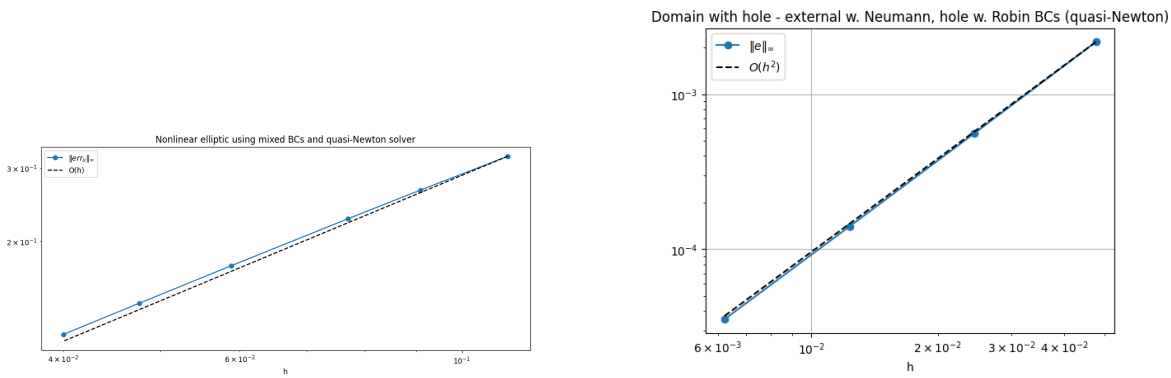
Another interesting phenomenon captured by some PDEs is the chemotactical movement described by the Keller-Segel model: solving it in the parabolic case Turing instabilities will appear. Keller and Segel modelled the attraction/repulsion between two species imposing that the flux of one species depends on the presence of the other giving rise to a nonlinear term of the form:$... + \nabla \cdot (u\nabla v) + ...$ As as an example for testing convergence we used the system

$$\begin{cases} \Delta u + \nabla \cdot (u\nabla v) = f_u \\ \Delta v = f_v \end{cases}$$



(a) (b)

Figure 3: Convergence plots: Poisson's equation.



(a) (b)

Figure 4: Convergence plots: cross-diffusion and domain with hole.

with known solutios $u_1 = sin(2\pi x), u_2 = sin(2\pi x)sin(2\pi y) u_3 = sin(2\pi x)sin(2\pi y)sin(2\pi z)$ in 1, 2 and 3 dimensions respectively. We discarded the naive approach to discretize the nonlinear term using the product rule expansion because it can generate instabilities. Hence, as suggested, we implemented a conservative discretization of the divergence of the flux in the middle points of our finite grid. This technique is implemented only in the two dimensional case following the notes. This method results in first order convergence since the term is discretized only in the interior points forgetting the boundary. We tried applying the ghost node strategy for Robin's BC to obtain second order convergence rate without success so we will proceed without it. The test was performed in the file testELL_nlin.ipynb where we also tested second order convergence of the Poisson's problem on a domain with a rectangular hole using Robin's BC as we will need it for our project. Results are shown in Figure 4.

1.3. Parabolic operators

We then explored the Method of Lines (MoL) to solve the diffusion equation:

$$u_t = \Delta u.$$

The RHS is discretized using the tecniques described in the preceding sections resulting in a semi-linear ODE: $u_t = A\overline{u}$. The ODEs system is then solved iteratively using one of the methods described in section 1.1 provided an initial condition $u(0, x)$. This works in every dimension but in the 3D case the large (often ill conditioned) matrices make difficult to solve the problem with a high degree of accuracy.

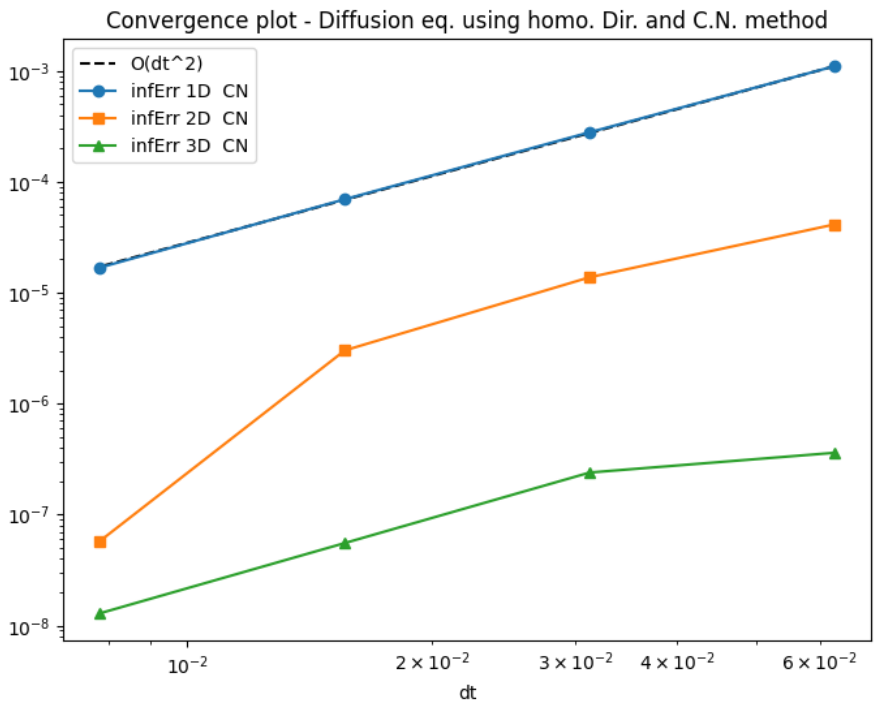


Figure 5: Convergence plot as time-grid refines for the diffusion equation.

The choice of the solver is crucial: being the problem derived from a PDE often stiff we tested our implicit solvers (impl_Euler, impl_CN) and some native solver (BDF). We tested convergence on a known solution in dimensions 1, 2 and 3 and results are shown in Figure 4.

1.4. Hyperbolic operators

We investigated some hyperbolic operators. Starting from a scalar advection equation

$$u_{tt} + au_x = 0$$

we first tested the upwind scheme which changes method with respect to the sign of a. We also tested stability using different Courant numbers and non-smooth initial conditions. We implemented higher order methods: the Lax-Friedrichs and the Lax-Wendroff testing again for convergence. Here instead of Dir./Rob. BCs we used periodic BC and we explored the case of one spatial dimension but with our implementation will be easy to work with higher dimensions in the future. In this case we tested convergence rate for the Lax-Friederichs scheme and results are shown in Figure 6. The generating code of this section and the next subsection is in the file testHYP.ipynb.

1.4.1 Burger's and wave equations

Finally, using our one dimensional operators, we implemented the inviscid Burgers' equation in conservative form

$$u_t + \left(\frac{u^2}{2}\right)_x = 0.$$

using periodic BCs. Our last experiment was to discretize the wave equation

$$u_{tt} = c\Delta u$$

in the one dimensional case with constant velocity c and homogeneous Dirichlet BCs. Some of the generated movies will be in the folder 'out'.

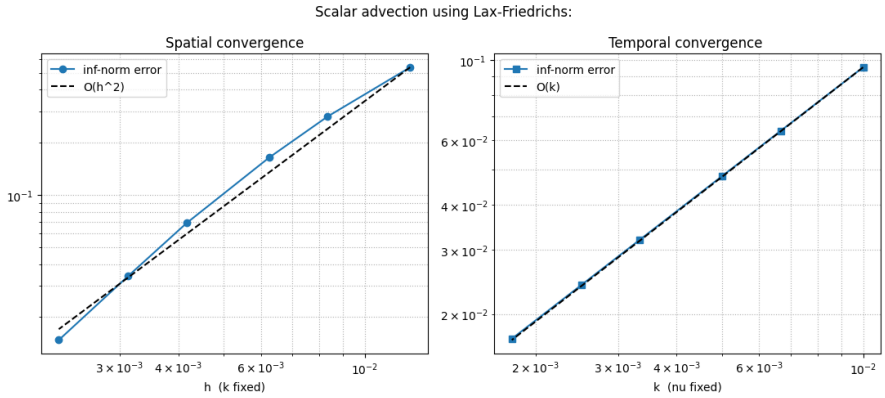


Figure 6: Convergence plot on scalar advection using Lax-Friederichs scheme.

2. Reaction-Diffusion Model in Alzheimer's disease

2.1. Model description

The model studied in our project stems from the study of the onset and progression of the Alzheimer's disease. It is a neurodegenerative disease and one model of the neuronal damage is amyloid cascade hypothesis. Alzheimer's disease has been identified as a protein misfolding disease caused by the accumulation of abnormally folded amyloid beta protein into amyloid plaques. Indeed in our model $u = u(x, y, t)$ represents the the concentration of amyloid proteins and $I = I(x, y, t)$ the concentration of interleukines (cytokines secreted by white blood cells that represents the inflammation degree). Here we think $t \geq 0$ and the spatial domain as a transverse section of the brain represented by a rectangular domain with two holes. The amyloid proteins become toxic only in their oligomeric form so the full model is a reaction-aggregation model using Smoluchowski coagulation equation by Cuperca et al. After some simplifications on the aggregation phenomenon we get the following diffusion- reaction model:

$$\begin{cases} u_t = \Delta u + \alpha I - \gamma u \\ I_t = d\Delta I - \chi \nabla \cdot (I \nabla u) + \frac{u}{1+u} - I. \end{cases} \tag{1}$$

The parameters of the model are: $\alpha = 1, d = 1.5, \chi = 1, \gamma = 5$.

2.2. Stationary Model - FD

In the file A_ell.ipynb we computed the finite difference solution on rectangular grids of $N_x \times N_y$ points of the described model. The boundary conditions are homogeneous Neumann both for u and I on the external boundary representing a zero normal flux. On the internal boundaries (the holes) we used Robin's BCs with parameters $\alpha_u = 1, \beta_u = 1, g_u = 0$ for u and $\alpha_I = d, \beta_I = 1, g_I = 0$. In the code we used different solvers for the FD approximation of the solution: the scipy.optimize nonlinear solver fsolve, a custom Newton method using the exact (analytically derived from the FD approximation of our operators) Jacobian matrix and a custom quasi-Newton method using the FD approximation of the Jacobian matrix. In our custom methods we tested some embedded iterative solvers investigated in section 1.2.1 concluding that GMRES was better that BICGSTAB in this case. We implemented custom stopping criteria using relative tolerances for inner iterations and we also used a stopping criteria based on the residual. In the custom Newton method we also added a damping parameter although smaller than the quasi-Newton case to prevent overshooting. The computed solutions using initial condition $(u, I) = (1, 1)$ are depicted in Figure 7. We see that the fsolve solution is the faster and the quasi-Newton is the slower one. Comparing the solutions with the one computed using MATLAB's PDEtoolbox and computing residuals we also see that the quasi-Newton method does not approximate the stationary case of our problem well, so we exclude it from the computational resource tests. Focusing on the first two solutions we see that the computational resources needed by our custom_newton() method are far higher in time but the memory resources needed are less than the fsolve case as shown in Table 3.

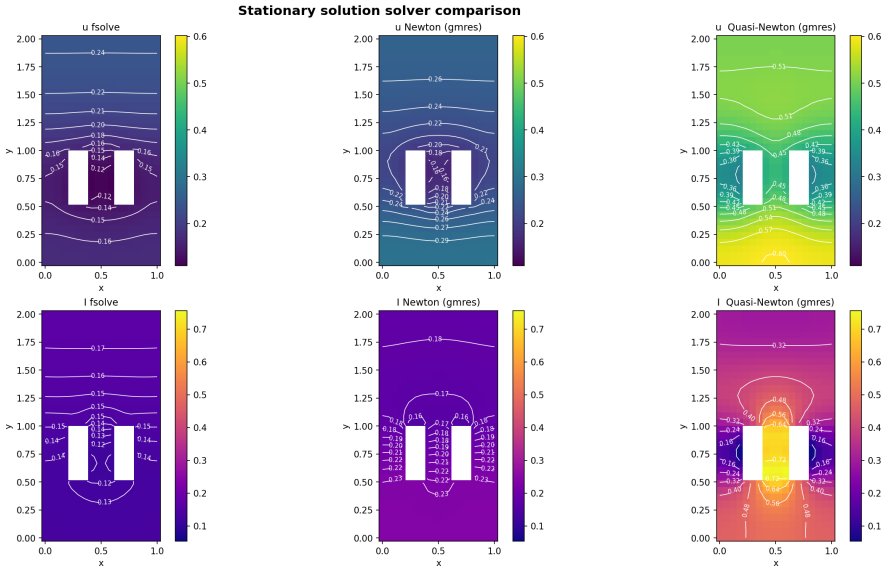
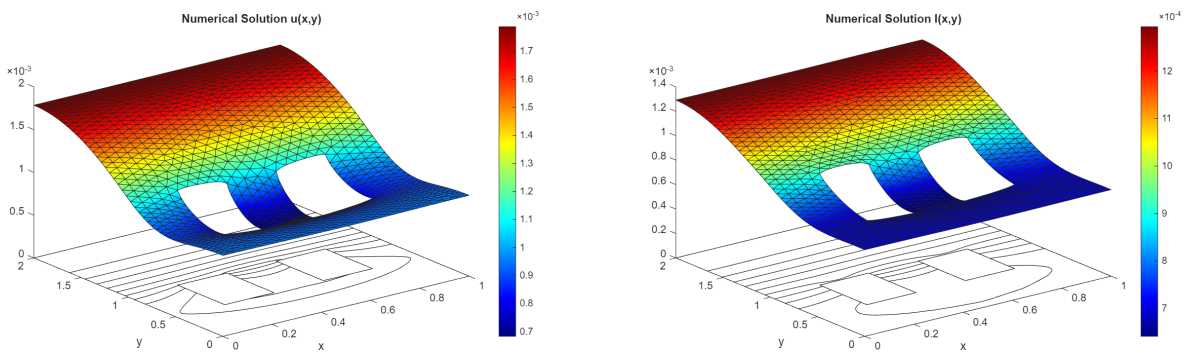


Figure 7: Comparison of FD solutions.

Meth.	$N_{grid-points}$	**Runtime (s)**	**Memory (MB)**
Newton	378	488.3	0.4
Newton	496	552.2	0.5
fsolve	378	2.5	5.4
fsolve	496	4.9	9.5

Table 3: Computational resources on project model using FD.



(a) (b)

Figure 8: Computed solutions using FEM.

The residual computation shows clearly that the best implementation methodology is the optimized scipy solver fsolve. The complete code that produces these results is in the folder called 'A', in the file A_ell.ipynb.

2.3. Stationary Model - Finite Elements Method (FEM)

In the file A_ell.m (inside the folder 'MAT') we implemented the FEM solution of our model using MATLAB's pdeToolbox. This solver uses FEM instead of FD meaning that it solves the weak form of our equations instead of discretizing the differential operators. The underlying mathematics is explained in the next section. This method is quite hard to implement, but the PDE Toolbox provides most of the required functionality. Here, as above, we plotted the solution and tested for variable mesh refinement. Some of the solutions are shown in Figure 8 and Table 4.

El. Order	N_{nodes}	Condition n.	Runtime (s)
lin	154	745.8	1.99
lin	534	989.9	1.81
lin	1150	675.62	2.50
quad	549	1246.2	2.96
quad	2005	1462.2	9.50
quad	4403	1126.7	29.19

Table 4: Computational resources on stationary model using FEM.

We note that this solver is considerably faster and more accurate than our custom solvers; also the fsolve strategy is slower and less accurate than FEM using linear elements but the difference becomes smaller.

2.4. Full Model - FEM and FD

Let's explain a bit of the FEM theory applied to our model. After multiplying it by a test function $v \in V$ and integrating it over the domain Ω our first equation:

$$\int_\Omega u_t v d\Omega = \int_\Omega \Delta u v d\Omega + \int_\Omega \alpha I v d\Omega - \int_\Omega \gamma u v d\Omega.$$

Now we apply the Green's formula:

$$-\int_\Omega \Delta u v d\Omega = \int_\Omega \nabla u \cdot \nabla v d\Omega - \int_{\partial\Omega} \frac{\partial u}{\partial \overrightarrow{\nu}} v d\gamma$$

and get:

$$\int_\Omega u_t v d\Omega + \int_\Omega \nabla u \cdot \nabla v d\Omega - \int_\Omega \alpha I v d\Omega + \int_\Omega \gamma u v d\Omega - \int_{\partial\Omega} \frac{\partial u}{\partial \overrightarrow{\nu}} v d\gamma = 0. \tag{2}$$

The same steps, used in the second equation this time, give us

$$\int_\Omega I_t v d\Omega + \int_\Omega \nabla I \cdot \nabla v d\Omega + \int_\Omega (\chi I) \nabla u \cdot \nabla v - \int_\Omega (\frac{u}{1+u} + I) v d\Omega - \int_{\partial\Omega} \frac{\partial I}{\partial \overrightarrow{\nu}} v d\gamma = 0. \tag{3}$$

Choosing an appropriate space for test functions (usually with linear or quadratic basis functions) we get the weak form of our problem: find $v \in V$ so that (2) and (3) are satisfied $\forall v \in V$. The computation of the method constants and the implementation of BCs are performed inside the pdeToolbox routine. After specifying the starting conditions we can solve the stationary version of our model. Some plots of the solution as time evolves using quadratic elements are shown in Figure 9. Both the u and I solutions get closer to 0 as t grows.

For the FD solution the file to look at is A_par.ipynp in which we implemented the Method of Lines using various solvers. We tested the Crank Nicolson method and the scipy's BDF method in solve_ivp. Here we plotted the solutions as movies and we see that the behaviour is very similar to that of the FEM method. Finally, in Figure 10 we show the computed residual over time.

3. Some implementation details and conclusions

Our experiments were performed on a machine using a Intel(R) Core(TM) i71165G7 processor.
From our experiments we discovered that the difference between MATLAB and python regarding accuracy and resource usage is quite subtle.
We can say that our custom methods were far slower than the ones optimized by the open source community so we couldn't reach the same accuracy in an acceptable amount of time. We conclude

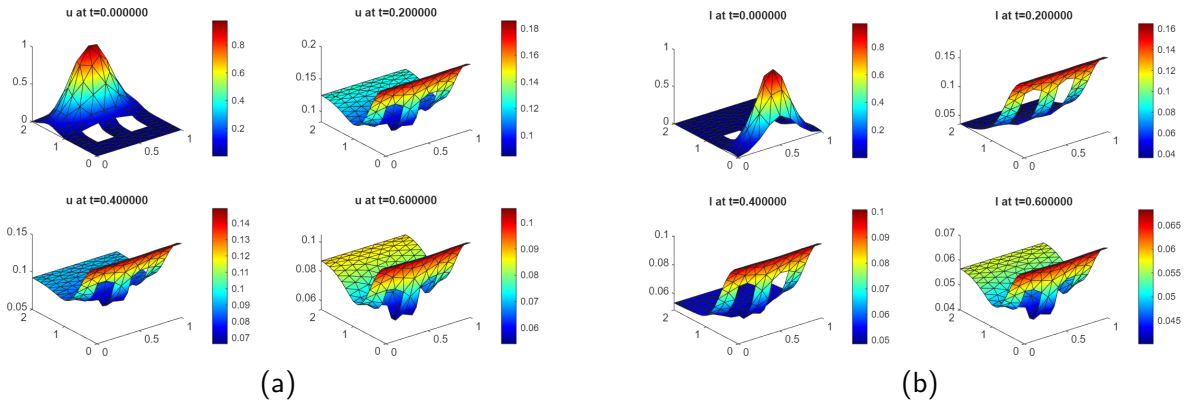


Figure 9: Computed solution: nonstationary model using FEM.

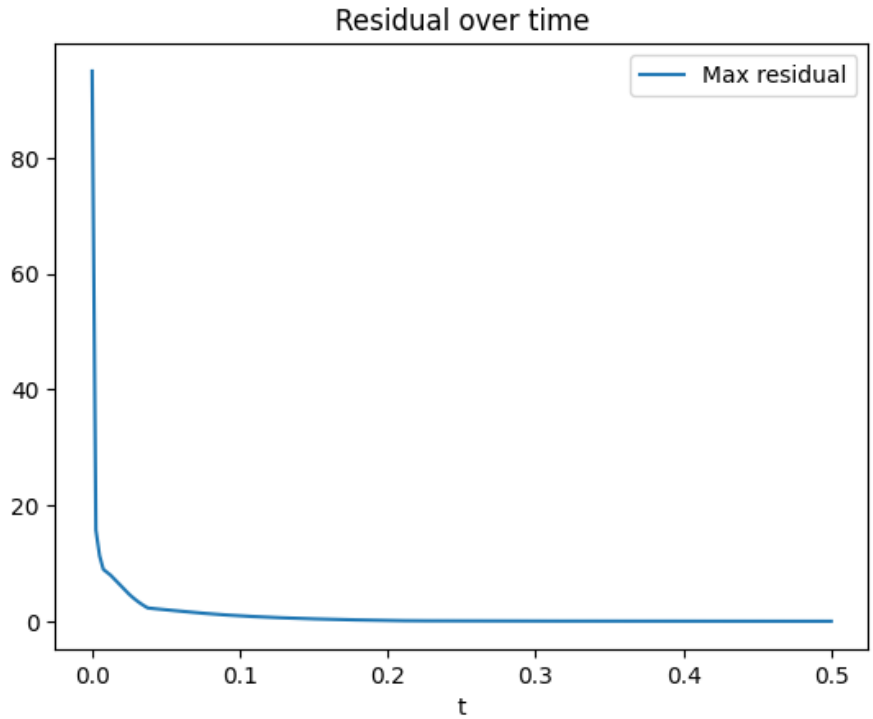


Figure 10: Residual vs time.

saying that the FEM method is far more fast and accurate but the naive FD approach is far easier to implement and can be usefull in simple cases though is better to use FEM for real-world applications.